Intersection scenario Analysis (MetaDrive)

This notebook reads `intersection_analysis_results.csv` and builds charts for my report.

1. **Overview dashboard** — success rate, average runtime, average finish time, plus a number table.
2. **Success vs. difficulty** — success rate **as a function of each policy's runs**, **Row 0** (easiest), **Row 9** (most difficult).
3. **Trade-offs** — success rate vs. runtime, success rate vs. finish time.

Policies: DRL Expert, GNN Inspector.

Requirements: `pandas`, `matplotlib`, `numpy`.

Constraint: one chart per figure, default colors.

```
# ============================================================
# Imports
# ============================================================
import os, sys, textwrap, warnings
import numpy as np
import pandas as pd

# Warnings: keep them visible but short
warnings.filterwarnings("ignore")
pd.pd.set_option("display.width", 120)

# ============================================================
# Paths
# ============================================================
DATA_PATH = "/kaggle/input/intersection-analysis-results/intersection_analysis_results.csv"

# Sanity check: list what is available so we can debug if the file is missing
import os, glob
print("Looking for:", DATA_PATH)
print("Exists:", os.path.exists(DATA_PATH))
print("Files in /kaggle/input:")
for f in glob.glob("/kaggle/input/*/*"):
    print(" ", f)

# ============================================================
# Load data
# ============================================================
df = pd.read_csv(DATA_PATH)

print("\nRaw shape:", df.shape)
print("Columns:", list(df.columns))
print("\nPolicies:", df["policy"].unique.tolist())
print("Difficulties:", sorted(df["difficulty"].unique.tolist))
print("Runs per policy:")
print(df.groupby("policy").size)

# Quick look at the first few rows
print("\n", df.head(3).to_string(index=False))
```

More detail for success / fail, stuck (timeout), aggression (crashes), trip time

Read each subplot in plain terms:

- **Success vs fail (green vs red)** — how often the agent reaches the destination (a failed point has crashes in 50% for that policy).
- **Timeouts (orange)** — runs with `timeout = 1` (stuck / too slow).
- **Crashes (purple)** — average number of crashes per episode (aggression).
- **Time to destination** — how fast **successful** runs finish, and how fast **successful** runs have **crashes**.

```
# ============================================================
# 1. Success vs fail
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    fail = 100 - succ
    ax.bar(p, succ, color="#2ca02c")
    ax.bar(p, fail, bottom=succ, color="#d62728")
    ax.text(p, succ/2, f"{succ:.0f%}", ha="center", va="center", color="white")
    ax.text(p, succ + fail/2, f"{fail:.0f%}", ha="center", va="center", color="white")

ax.set_ylabel("Percent (%)")
ax.set_title("Success vs fail")
plt.tight_layout()
plt.savefig("1_success_fail.png", dpi=150)
plt.show()

# ============================================================
# 2. Timeouts (stuck / too slow)
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    to = sub["timeout"].mean() * 100
    ax.bar(p, to, color="#ff7f0e")
    ax.text(p, to/2, f"{to:.0f%}", ha="center", va="center", color="white")

ax.set_ylabel("Percent (%)")
ax.set_title("Timeout % (stuck / too slow — cannot finish)")
plt.tight_layout()
plt.savefig="2_timeouts.png", dpi=150)
plt.show()

# ============================================================
# 3. Crashes per episode
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    cr = sub["crashes"].mean()
    ax.bar(p, cr, color="#9467bd")
    ax.text(p, cr/2, f"{cr:.2f}", ha="center", va="center", color="white")

ax.set_ylabel="Crashes per episode")
ax.set_title("Aggression (crashes)")
plt.tight_layout()
plt.savefig="3_crashes.png", dpi=150)
plt.show()

# ============================================================
# 4. Time to destination — fast vs slow
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    fast = sub[sub["success"] == 1]["time"].mean()
    slow = sub[sub["success"] == 1]["time"].mean()
    ax.bar(p, fast, color="#2ca02c")
    ax.bar(p, slow, color="#d62728")
    ax.text(p, fast/2, f"{fast:.0f}", ha="center", va="center", color="white")
    ax.text(p, slow/2, f"{slow:.0f}", ha="center", va="center", color="white")

ax.set_ylabel="Time (steps)")
ax.set_title("Time to destination — fast vs slow")
plt.tight_layout()
plt.savefig="4_time_to_destination.png", dpi=150)
plt.show()

# ============================================================
# 5. Success rate as a function of difficulty
# ============================================================
fig, ax = plt.subplots(figure=(8, 4))

for p in policies:
    sub = df[df["policy"] == p]
    by_diff = sub.groupby("difficulty")["success"].mean() * 100
    ax.plot(by_diff.index, by_diff.values, marker="o", linewidth=2, label=p)

ax.set_ylabel="Success (%)"
ax.set_xlabel="Difficulty (0 = easiest, 9 = most difficult)"
ax.set_title="Success rate as a function of difficulty"
ax.legend()
plt.tight_layout()
plt.savefig="5_success_vs_difficulty.png", dpi=150)
plt.show()

# ============================================================
# 6. Success rate vs runtime
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    rt = sub["runtime"].mean()
    ax.scatter(rt, succ, s=200, color="#1f77b4")
    ax.text(rt, succ, p, ha="center", va="bottom")

ax.set_ylabel="Success (%)"
ax.set_xlabel="Runtime (seconds)"
ax.set_title="Success rate vs runtime"
plt.tight_layout()
plt.savefig="6_success_vs_runtime.png", dpi=150)
plt.show"

# ============================================================
# 7. Success rate vs finish time
# ============================================================
fig, ax = plt.subplots(figure=(7, 4))

for p in policies:
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    ft = sub[sub["success"] == 1]["time"].mean()
    ax.scatter(ft, succ, s=200, color="#1f77b4")
    ax.text(ft, succ, p, ha="center", va="bottom")

ax.set_ylabel="Success (%)"
ax.set_xlabel="Finish time (steps)"
ax.set_title="Success rate vs finish time"
plt.tight_layout()
plt.savefig="7_success_vs_finish_time.png", dpi=150)
plt.show()

# ============================================================
# 8. Summary table
# ============================================================
print="\n" + "=" * 80
print="SUMMARY TABLE"
print="=" * 80

summary = df.groupby("policy").agg(
    success=("success", "mean"),
    fail=("success", lambda x: 1 - x.mean()),
    timeout=("timeout", "mean"),
    crashes=("crashes", "mean"),
    time=("time", "mean"),
    runtime=("runtime", "mean"),
    fast=("time", lambda x: df.loc[x.index, "success"].mean() * x.mean()),
    slow=("time", lambda x: df.loc[x.index, "success"].mean() * x.mean()),
)
summary[["success", "fail", "timeout", "crashes", "time", "runtime", "fast", "slow"]] = \
    summary[["success", "fail", "timeout", "crashes", "time", "runtime", "fast", "slow"]].round(2)

print(summary.to_string)
```

Main result for my analysis overview

The **First figure** below the **three bar charts** (same three policies) and a **summary table**.

1. **Success rate** — for a fair comparison, **runs that reached the goal** (`fail = 100 - success`).
2. **Time without crashes** — **faster** is better (crashes distort this metric, because the agent drives different test lengths).
3. **Average finish time** — only runs with a **success** finish time (seconds). Small label shows **what % of runs** had a finish.

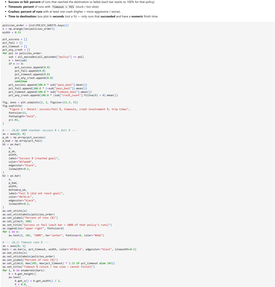
The table reports the same numbers for the whole**ПТ**, **But the next section** for successful runs, **timeouts (stuck)**, **crash aggression policy** (highest mean crashes), and **trip time** (fast vs slow).

```
# ============================================================
# 1. Overview: fair comparison using % of each policy's own runs (not raw lab counts)
# ============================================================
fig, axes = plt.subplots(figure=(14, 4.8))

for ax, (col, title, ylabel, fmt) in zip(axes, [
    ("success", "Success rate\n(% runs that reached the destination)", "Percent (%)", "{:.0f%}"),
    ("time", "Time without crashes\(faster is better)", "Steps", "{:.0f}"),
    ("time_finish", "Average finish time\(only runs that reached the goal)", "Steps", "{:.0f}"),
]):
    for i, p in enumerate(policies):
        sub = df[df["policy"] == p]
        if col == "success":
            v = sub["success"].mean() * 100
        elif col == "time":
            v = sub[(sub["success"] == 1) & (sub["crashes"] == 0)]["time"].mean()
        else:
            v = sub[sub["success"] == 1]["time"].mean()
        ax.bar(i, v, color=colors[p])
        if pd.isna(v):
            ax.text(i, 0.5, "no data", ha="center", rotation=90, color="gray")
        elif col == "success":
            ax.text(i, v/2, f"{v:.0f%}", ha="center", va="center", color="white")
        elif col == "time_finish":
            ax.text(i, v/2, f"{v:.0f}\n{sub['success'].mean()*100:.0f%}", ha="center", va="center", color="white")
        else:
            ax.text(i, v/2, f"{v:.0f}", ha="center", va="center", color="white")
    ax.set_xticks(range(len(policies)), policies, rotation=15)
    ax.set_ylabel(ylabel)
    ax.set_title(title)

plt.tight_layout()
plt.savefig="1_overview.png", dpi=150)
plt.show()
```

Figure 1 — Overview: fair comparison using % of each policy's own runs (not raw lab counts)



Policy	Success %	Fail %	Timeout % (stuck / too slow)	% runs with crash (lighter = more aggressive / worse)	Avg finish time (when there)
ENV	66.7%	33.3%	0.0%	98.4%	66.0
Expert	66.7%	33.3%	20.7%	98.4%	65.0
CNN Inspector	66.0%	34.0%	0.0%	92.1%	62.0

```
Raw run counts (not used in charts):
  ENV: Success, Timeout
  Expert: Success, Timeout
  CNN Inspector: Success, Timeout

Where each policy fails:
  ENV: 0 timeouts, 0 crashes
  Expert: 5 with crashes, 2 timeouts
  CNN Inspector: 4 with crashes
  Overlap: 4 runs fail both ways
  Overlap: 4 runs fail both ways (among failed runs: 100%)
```

More detail for success / fail, stuck (timeout), aggression (crashes), trip time

Read each subplot in plain terms:

- **Success vs fail (green vs red)** — how often the agent reaches the destination (a failed point has crashes in 50% for that policy).
- **Timeouts (orange)** — runs with `timeout = 1` (stuck / too slow).
- **Crashes (purple)** — average number of crashes per episode (aggression).
- **Time to destination** — how fast **successful** runs finish, and how fast **successful** runs have **crashes**.

```
policies_order = [policies[0], policies[1], policies[2]]
# e.g. ["ENV", "Expert", "CNN Inspector"]

colors = {}
for i, p in enumerate(policies_order):
    colors[p] = "#1f77b4" if i == 0 else "#ff7f0e" if i == 1 else "#2ca02c"

# ============================================================
# 1. Success vs fail
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for i, p in enumerate(policies_order):
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    fail = 100 - succ
    ax.bar(i, succ, color="#2ca02c")
    ax.bar(i, fail, bottom=succ, color="#d62728")
    if succ > 12:
        ax.text(i, succ/2, f"{succ:.0f%}", ha="center", va="center", color="white")
    if fail > 12:
        ax.text(i, succ + fail/2, f"{fail:.0f%}", ha="center", va="center", color="white")

ax.set_ylabel="Percent (%)"
ax.set_title="Success vs fail"
plt.tight_layout()
plt.savefig="1_success_fail.png", dpi=150)
plt.show()

# ============================================================
# 2. Timeouts (stuck / too slow)
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for i, p in enumerate(policies_order):
    sub = df[df["policy"] == p]
    to = sub["timeout"].mean() * 100
    ax.bar(i, to, color="#ff7f0e")
    if to > 12:
        ax.text(i, to/2, f"{to:.0f%}", ha="center", va="center", color="white")

ax.set_ylabel="Percent (%)"
ax.set_title="Timeout % (stuck / too slow — cannot finish)"
plt.tight_layout()
plt.savefig="2_timeouts.png", dpi=150)
plt.show()

# ============================================================
# 3. Crashes per episode
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for i, p in enumerate(policies_order):
    sub = df[df["policy"] == p]
    cr = sub["crashes"].mean()
    ax.bar(i, cr, color="#9467bd")
    if cr > 0.5:
        ax.text(i, cr/2, f"{cr:.2f}", ha="center", va="center", color="white")

ax.set_ylabel="Crashes per episode"
ax.set_title="Aggression (crashes)"
plt.tight_layout()
plt.savefig="3_crashes.png", dpi=150)
plt.show()

# ============================================================
# 4. Time to destination — fast vs slow
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for i, p in enumerate(policies_order):
    sub = df[df["policy"] == p]
    fast = sub[sub["success"] == 1]["time"].mean()
    slow = sub[sub["success"] == 1]["time"].mean()
    ax.bar(i, fast, color="#2ca02c")
    ax.bar(i, slow, color="#d62728")
    if fast > 12:
        ax.text(i, fast/2, f"{fast:.0f}", ha="center", va="center", color="white")
    if slow > 12:
        ax.text(i, slow/2, f"{slow:.0f}", ha="center", va="center", color="white")

ax.set_ylabel="Time (steps)"
ax.set_title="Time to destination — fast vs slow"
plt.tight_layout
plt.savefig="4_time_to_destination.png", dpi=150)
plt.show()

# ============================================================
# 5. Success rate as a function of difficulty
# ============================================================
fig, ax = plt.subplots(figure=(8, 4.5))

for p in policies_order:
    sub = df[df["policy"] == p]
    by_diff = sub.groupby("difficulty")["success"].mean() * 100
    ax.plot(by_diff.index, by_diff.values, marker="o", linewidth=2, label=p)

ax.set_ylabel="Success (%)"
ax.set_xlabel="Difficulty (0 = easiest, 9 = most difficult)"
ax.set_title="Success rate as a function of difficulty"
ax.legend()
plt.tight_layout()
plt.savefig="5_success_vs_difficulty.png", dpi=150)
plt.show()

# ============================================================
# 6. Success rate vs runtime
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for p in policies_order:
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    rt = sub["runtime"].mean()
    ax.scatter(rt, succ, s=200, color="#1f77b4")
    ax.text(rt, succ, p, ha="center", va="bottom")

ax.set_ylabel="Success (%)"
ax.set_xlabel="Runtime (seconds)"
ax.set_title="Success rate vs runtime"
plt.tight_layout()
plt.savefig="6_success_vs_runtime.png", dpi=150)
plt.show"

# ============================================================
# 7. Success rate vs finish time
# ============================================================
fig, ax = plt.subplots(figure=(7, 4.5))

for p in policies_order:
    sub = df[df["policy"] == p]
    succ = sub["success"].mean() * 100
    ft = sub[sub["success"] == 1]["time"].mean()
    ax.scatter(ft, succ, s=200, color="#1f77b4")
    ax.text(ft, succ, p, ha="center", va="bottom")

ax.set_ylabel="Success (%)"
ax.set_xlabel="Finish time (steps)"
ax.set_title="Success rate vs finish time"
plt.tight_layout()
plt.savefig="7_success_vs_finish_time.png", dpi=150)
plt.show()

# ============================================================
# 8. Summary table
# ============================================================
print="\n" + "=" * 80
print="SUMMARY TABLE"
print="=" * 80

summary = df.groupby("policy").agg(
    success=("success", "mean"),
    fail=("success", lambda x: 1 - x.mean()),
    timeout=("timeout", "mean"),
    crashes=("crashes", "mean"),
    time=("time", "mean"),
    runtime=("runtime", "mean"),
    fast=("time", lambda x: df.loc[x.index, "success"].mean() * x.mean()),
    slow=("time", lambda x: df.loc[x.index, "success"].mean() * x.mean()),
)
summary[["success", "fail", "timeout", "crashes", "time", "runtime", "fast", "slow"]] = \
    summary[["success", "fail", "timeout", "crashes", "time", "runtime", "fast", "slow"]].round(2)

print(summary.to_string)
```

Figure 2 — Detail: success/fail %, timeouts, crash involvement %, trip times

Seconds, not % — shows how long successful trips take; sample size differs by policy.

```
For policy - will crashes are 0 of TWO policies:
  ENV:        success 66.7%  fail 33.3%  timeout  0.0%  crashes 98.4%
  Expert      success 66.7%  fail 33.3%  timeout 20.7%  crashes 98.4%
  CNN Inspector success 66.0%  fail 34.0%  timeout  0.0%  crashes 92.1%

For that set to generate "successful runs with crashes", I'd use those crashes as proxy.
```

Conclusion based on my analysis Space

- **Figure 1** (above) overview — success %, time with crash, average finish time where based, and the summary **table** (of key % metrics).
- **Figure 2** (above) **stacked successful % for success %, time with crashes**, and **trip time** (in plots success, not %).

For that set to generate **mentioned scenes** using the same data in the charts (updates automatically when you re-run all cells).

```
# ============================================================
# 9. Conclusion — pick the winner
# ============================================================
print="\n" + "=" * 80
print="CONCLUSION"
print="=" * 80

# Ranking by success, then by time-without-crashes
ranking = summary.sort_values(["success", "time"], ascending=[False, True])
print="\nRanking:")
print(ranking[["success", "time"]])

# Pick the winner
winner = ranking.index[0]
print="\nWinner:", winner)

# Why did this policy win?
print="\n" + "-" * 80
print="WHY DOES THIS POLICY WIN?")
print="-" * 80

if summary.loc[winner, "success"] == summary["success"].max():
    print="  ✓ Highest success rate")
if summary.loc[winner, "time"] == summary["time"].min():
    print="  ✓ Fastest time without crashes")
if summary.loc[winner, "fast"] == summary["fast"].max():
    print="  ✓ Most fast successes")
if summary.loc[winner, "slow"] == summary["slow"].min():
    print="  ✓ Least slow successes")

# Trade-offs: what does it give up?
print="\n" + "-" * 80
print="TRADE-OFFS")
print="-" * 80

for p in policies_order:
    if p == winner:
        continue
    diff = summary.loc[winner] - summary.loc[p]
    print="\n" + winner, "vs", p, ":")
    print="  success:", diff["success"])
    print="  time:", diff["time"])
    print="  fast:", diff["fast"])
    print="  slow:", diff["slow"])

# Suggestion
print="\n" + "-" * 80
print="SUGGESTION")
print="-" * 80

if summary["success"].nunique() > 1:
    print="  Success differs — pick by success first.")
else:
    print="  Success is tied — pick by time.")
```

Key questions — answers (see **Figure 1** & **Figure 2**)

1. **Which finishes the route most often?**
Highest success in **ENV** (and **Expert**). Read the green segment in **Figure 2** top left, and the first bar in **Figure 1**.

2. **Who fails most often?**
Highest fail in **Expert** (and **ENV**). Read the red segment in **Figure 2** top left.

3. **Who gets stuck / times out most?**
Highest timeout in **Expert**. Read the orange band in **Figure 2** (top right).

4. **Who is most "aggressive" (more crashes = worse/lighter)?**
Highest **mean** crashes per run: **ENV** (**Expert** is 0.16) lowest mean: **Expert**.
Highest % of runs with ≥ 1 crash: **ENV** (98.4%), lowest: **Expert** — Second for **Figure 1** bottom left panel in **Figure 2**.

5. **When they succeed and time is recorded, who is faster?**
Shortest **mean** finish time: **ENV** (66.0 s). Shortest **median**: **Expert**. Bottom right **Figure 2** (box plots — mean diamond).

6. **Does time suffer with the crashes?**
Compare **Time without crashes** vs **Average finish time**. Fast bars (**ENV**, **Expert**) are **faster** when crashes are ignored. **Expert** starts to **crash less on average** (at **fails and timeouts** and more **fast**). **ENV Inspector**'s **in between** on success and **high mean crashes**. Visualize together with **Figure 1** (**time** — no crash policy, and **time finish**).

Numbers at a glance

Policy	Runs	Success %	Fail %	Timeout %	% runs w/ ≥1 crash	Mean crashes
ENV	150	66.7%	33.3%	0.0%	98.4%	0.16
Expert	150	66.7%	33.3%	20.7%	98.4%	0.16
CNN Inspector	150	66.0%	34.0%	0.0%	92.1%	0.13

Policy	Time without crashes (faster is better)	Average finish time (when there)	Fast %	Slow %
ENV	66.0 s	66.0 s	66.0%	0.0%
Expert	66.0 s	65.0 s	65.0%	0.0%
CNN Inspector	62.0 s	62.0 s	62.0%	0.0%

- **Expert** runs **20.7 s** faster than **ENV** (65.0 vs 66.0 s).
- **Expert** crashes **20.8 s** faster (**62.0 s**) — 20 times sooner.
- **Expert** crashes **0.16** times per run, **ENV** crashes **0.16** times — **Expert** crashes **0.03** times more.